

# C3 - Intelligence pour la robotique: TP6 – Topologies des réseaux de neurones artificiels

Justin Ferdinand

## I LES TOPOLOGIES MULTICOUCHES

### I.A EXERCICE 1 : ÉTUDE D'UN PERCEPTRON SIMPLE

#### I.A.1 QUESTION 1 : INFLUENCE D'UN POIDS UNIQUE

L'objectif de cette première question est d'observer l'effet d'un poids  $w_1 = -0.5$  sur la sortie d'un neurone avec fonction d'activation de saturation entre  $-1$  et  $1$ .

Pour une entrée  $x_1 \in [-2.0, 2.0]$  avec un pas de  $0.2$ , la sortie est calculée par :

$$y_1 = \text{sat}(w_1 \cdot x_1) \quad 1.$$

où  $\text{sat}(x) = \text{clip}(x, -1, 1)$ .

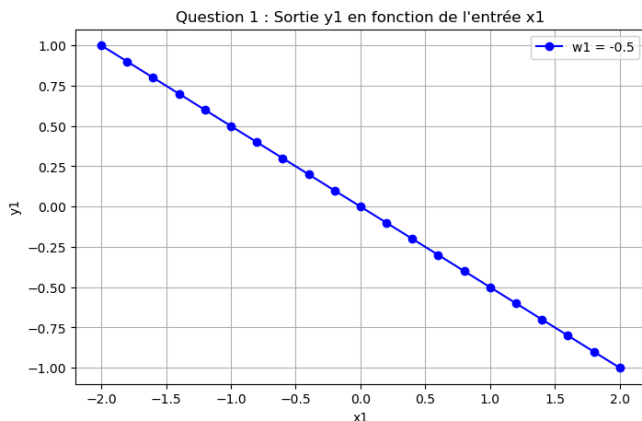


Figure 1: Sortie  $y_1$  en fonction de l'entrée  $x_1$  avec  $w_1 = -0.5$

**Observations :** La Figure 1 montre une relation linéaire dans la zone non saturée. Le poids négatif inverse la relation entrée-sortie. Les bornes de saturation ( $-1$  et  $1$ ) sont atteintes pour des valeurs d'entrée éloignées de zéro.

#### I.A.2 QUESTION 2 : VARIATION DES POIDS

Cette question explore l'influence de 15 valeurs de poids différentes dans l'intervalle  $[-1.0, 1.0]$  sur la même entrée.

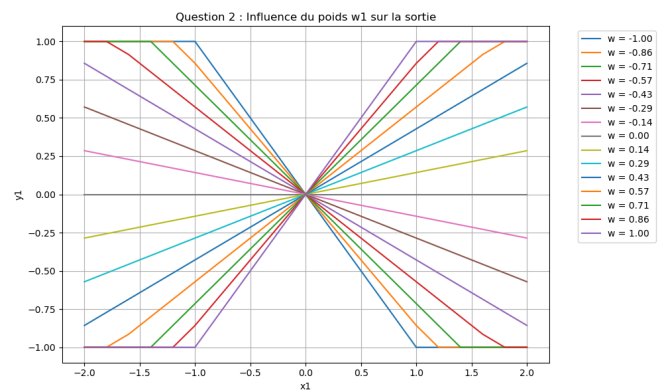


Figure 2: Influence du poids  $w_1$  sur la sortie

La Figure 2 illustre plusieurs comportements :

- Dans les zones non saturées, la relation est linéaire :  $y = w \cdot x$
- La pente de la fonction dépend directement de la valeur absolue du poids
- Le signe du poids détermine le sens de la relation (positive ou négative)
- Plus  $|w|$  est grand, plus rapidement la saturation est atteinte

#### I.A.3 QUESTION 3 : RÉSEAU MULTICOUCHE À DEUX COUCHES

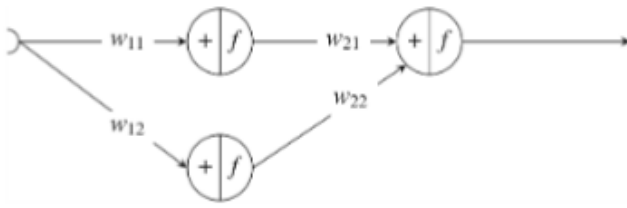


Figure 3: Architecture du réseau à deux couches étudié

Cette question introduit un réseau à deux couches avec deux neurones cachés. L'architecture visible sur la Figure 3 est la suivante :

- **Couche cachée** : deux neurones recevant la même entrée  $x_1$
- **Couche de sortie** : un neurone combinant les sorties des neurones cachés

Les poids utilisés sont :

$$w_{\{11\}} = 1.0, \quad w_{\{12\}} = 0.5 \quad (\text{entrée} \rightarrow \text{cachée})$$

$$w_{\{21\}} = 3.0, \quad w_{\{22\}} = -1.0 \quad (\text{cachée} \rightarrow \text{sortie})$$

Les équations sont :

$$h_1 = \text{sat}(w_{\{11\}} \cdot x_1)$$

$$h_2 = \text{sat}(w_{\{12\}} \cdot x_1)$$

$$y = \text{sat}(w_{\{21\}} \cdot h_1 + w_{\{22\}} \cdot h_2)$$

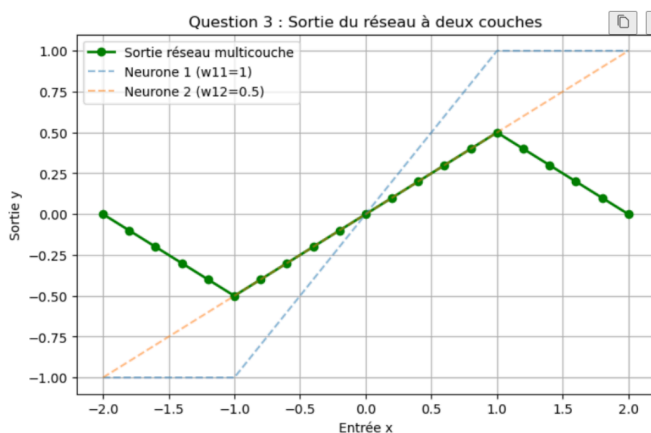


Figure 4: Sortie du réseau à deux couches

La Figure 4 montre une fonction non-linéaire plus complexe que les sorties individuelles des neurones. La sortie finale n'est pas monotone :

- Pour  $x < -1$  : le premier neurone sature à  $-1$ , le second est dans sa zone linéaire. La sortie est dominée par le second neurone.
- Pour  $-1 \leq x \leq 1$  : les deux neurones sont dans leur zone linéaire. La combinaison est une somme pondérée de fonctions linéaires.
- Pour  $x > 1$  : le premier neurone sature à  $1$ . La sortie devient la différence entre une constante et la fonction du second neurone.

En augmentant  $w_{\{21\}}$  à  $3.0$  (voir Figure 5), l'influence du premier neurone est amplifiée, créant une pente plus raide dans les zones où il est actif.

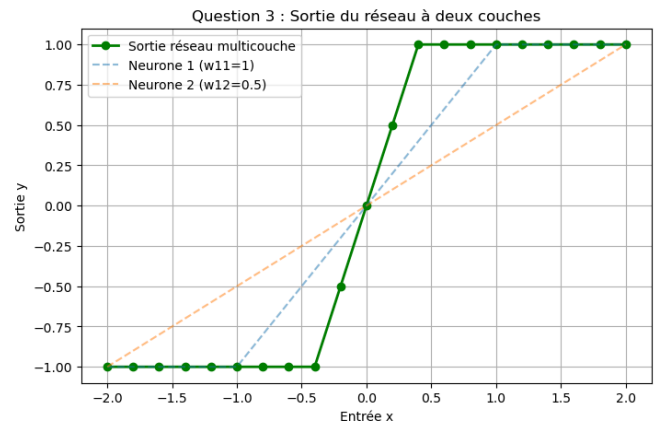


Figure 5: Sortie du réseau avec  $w_{\{21\}} = 3.0$  (influence amplifiée)

**Conclusion** : Les réseaux multicouches permettent de créer des fonctions complexes et non-monotones à partir de combinaisons de fonctions simples (linéarité + saturation). La variation des poids modifie l'influence relative de chaque neurone sur la sortie globale.

## I.B EXERCICE D'APPLICATION ROBOTIQUE

### I.B.1 DÉMARCHE SUIVIE

Pour concevoir un réseau capable d'éviter des obstacles avec la contrainte de reculer si l'objet est détecté par les deux capteurs, je me suis inspiré du TP précédent sur les perceptrons. L'objectif est que

chaque capteur influe sur le moteur opposé pour provoquer un virage à l'opposé de l'obstacle.

L'implantation réalisée utilise des valeurs de capteurs normalisées entre 0 et 1. La fonction d'activation utilisée pour tous les neurones est une fonction de saturation limitant l'intervalle de sortie à  $[-1, 1]$ .

### I.B.2 MODÈLE PROPOSÉ

Dans cette partie, j'utilise le modèle de réseau multicouche à deux couches illustré par la Figure 6.

- Les neurones d'entrée  $x_1$  et  $x_2$  correspondent respectivement aux capteurs avant-gauche et avant-droit.
- La couche cachée ( $h_1, h_2$ ) sert de relais pour les valeurs normalisées des capteurs.
- La couche de sortie ( $y_1, y_2$ ) pilote les moteurs (gauche et droit). Elle combine un biais positif pour avancer et des poids négatifs croisés pour reculer ou éviter un obstacle.

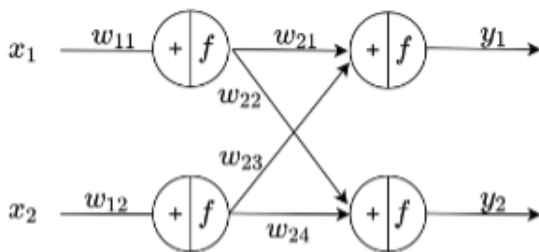


Figure 6: Modèle du réseau servant à l'évitement d'obstacles

### I.B.3 POIDS DU RÉSEAU

Les poids ont été déterminés logiquement pour satisfaire les conditions du comportement souhaité:

1. **Couche Entrée → Cachée** (chaque capteur a le même poids) :

$$w_{\{11\}} = 1.0, \quad w_{\{12\}} = 1.0 \quad 4.$$

2. **Couche Cachée → Sortie** :

$$w_{\{21\}} = 0.0, \quad w_{\{24\}} = 0.0 \quad 5.$$

(le capteur n'influence pas son propre capteur)

$$w_{\{22\}} = -2.0 \quad 6.$$

(Le capteur gauche  $h_1$  influence le moteur droit  $y_2$ )

$$w_{\{23\}} = -2.0 \quad 7.$$

(Le capteur droit  $h_2$  influence le moteur gauche  $y_1$ )

3. **Biais** :

$$b = 1.0 \quad 8.$$

(Injecté à la couche de sortie pour définir la vitesse par défaut)

### I.B.4 OBSERVATIONS LORS DE LA VALIDATION EXPÉRIMENTALE

Le comportement observé du robot lors des simulations montre que son réseau de neurones répond correctement à la configuration des murs du labyrinthe. Thymio est capable d'avancer en ligne droite et de tourner lorsque des angles sont détectés. De plus, s'il arrive face à un mur, thymio s'arrête et recule tout en tournant légèrement pour reprendre la route. Durant la simulation, j'ai ajouté des logs montrant les sorties  $y_1$  et  $y_2$  du réseau. On peut ainsi voir ces valeurs à chaque instant, confirmant la réaction du réseau à la situation courante.

En modifiant  $w_{\{21\}}$  et  $w_{\{24\}}$  à 1, j'ai pu observer que le robot ne réagissait pas forcément moins aux obstacles malgré le fait qu'un capteur influence son propre moteur. Cependant, la trajectoire de celui-ci était moins précise. En revanche, en mettant ces deux poids à  $-1$ , j'ai pu obtenir l'effet recherché de laisser le robot tourner davantage sur lui-même pour éviter un obstacle. Ainsi, chaque capteur influençait également son moteur de manière opposée pour renforcer l'évitement. Cela se voit en simulation lorsque les deux valeurs de  $y$  varient.

Pour éviter certains obstacles, j'ai dû utiliser les capteurs latéraux plus éloignés. Il arrivait régulièrement que les capteurs avant gauche et avant droit ne détectent plus l'obstacle.

Le code et la vidéo de la simulation sont disponibles dans le dossier Partie 1 du TP6.

## II MÉMOIRE - RÉSEAUX RÉCURRENTS

Dans cette seconde partie, le tp aborde l'implémentation de réseaux récurrents. L'intérêt principal est d'introduire une dépendance temporelle : la sortie du réseau ne dépend plus uniquement de l'instantané des capteurs à l'instant  $t$ , mais aussi de l'état précédent du système (mémoire à  $t - 1$ ). Cela permettra de garder en mémoire la détection d'un obstacle et de corriger le problème de perte d'information rencontré dans la partie précédente.

### II.A EXERCICE 2 : RÉSEAU RÉCURRENT SIMPLE

#### II.A.1 PRINCIPE ET MODÈLE

L'objectif de cet exercice est d'observer l'effet d'une connexion récurrente sur le comportement temporel d'un neurone. Le modèle utilisé est le suivant :

$$y(t) = \text{sat}(w_{\text{in}} \cdot x(t) + w_{\text{rec}} \cdot y(t - 1))$$

où :

- $x(t)$  est l'entrée au temps  $t$
- $y(t)$  est la sortie au temps  $t$
- $y(t - 1)$  est la sortie au temps précédent (mémoire)
- $w_{\text{in}}$  est le poids de l'entrée (fixé à 0.5)
- $w_{\text{rec}}$  est le poids récurrent
- $\text{sat}(x) = \text{clip}(x, 0, 1)$  est la fonction de saturation

L'entrée est un signal échelon qui s'active à  $t = 5$  avec une valeur de 0.3.

#### II.A.2 CAS TESTÉS

Deux configurations ont été testées pour observer différents comportements de mémoire :

**Cas (a) : Poids récurrent  $> 1$  ( $w_{\text{rec}} = 1.2$ )**

- La rétroaction amplifie le signal à chaque itération
- Comportement de mémorisation forte
- Le neurone tend à rester actif une fois activé

**Cas (b) : Poids récurrent dans  $[0, 1]$  ( $w_{\text{rec}} = 0.8$ )**

- Le système se stabilise à une valeur d'équilibre
- Comportement de filtre passe-bas ou de mémoire à court terme
- Le système "oublie" progressivement l'historique

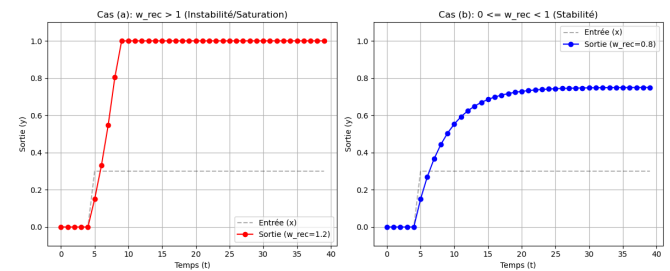


Figure 7: Comportement du neurone récurrent selon le poids  $w_{\text{rec}}$

**Conclusion :** Le poids récurrent contrôle la persistance de la mémoire. Un poids  $> 1$  crée une mémoire permanente (instable), tandis qu'un poids  $< 1$  crée une mémoire qui s'estompe progressivement (stable). Ce mécanisme est fondamental pour créer des comportements temporels dans les réseaux de neurones.

### II.B EXERCICE D'APPLICATION ROBOTIQUE

#### II.B.1 IMPLANTATION PROPOSÉE

Le modèle utilisé est un réseau monocouche avec connexions récurrentes. Il prend en entrée les capteurs (Gauche, Avant, Droit) et contrôle les deux moteurs.

Les équations d'activation choisies sont :

$$y_1(t) = (w_{\{11\}}x_L + w_{\{21\}}x_F + w_{\{31\}}x_R + w_4y_1(t-1) + b)$$

$$y_2(t) = (w_{\{12\}}x_L + w_{\{22\}}x_F + w_{\{32\}}x_R + w_5y_2(t-1) + b)$$

où  $b$  est le biais de marche avant et les poids  $w$  ceux présents sur la Figure 8

Les sorties  $y_1$  (gauche) et  $y_2$  (droit) sont réinjectées à l'entrée avec les poids  $w_4$  et  $w_5$ .

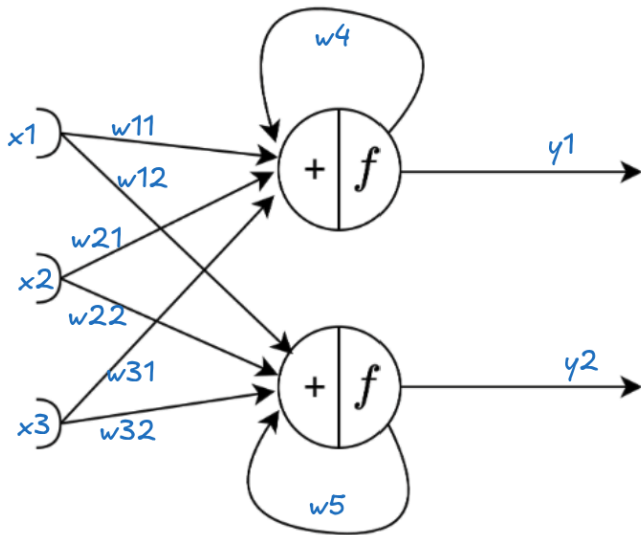


Figure 8: Modèle de réseau récurrent implémenté

## II.C MISE EN ŒUVRE

Pour tester l'effet de la mémoire, j'ai modifié le parcours de simulation pour inclure une section plus large avec un obstacle central. Cela permet d'observer comment le robot réagit lorsqu'il perd temporairement la détection de l'obstacle. Le nouveau parcours est visible sur la Figure 9.

J'ai utilisé plusieurs configurations de poids récurrents pour observer différents comportements de mémoire. J'ai ainsi testé des valeurs de  $w_4$  et  $w_5$  à 0.5, 1.0 et 1.5 pour voir comment cela affecte la persistance de l'activation.

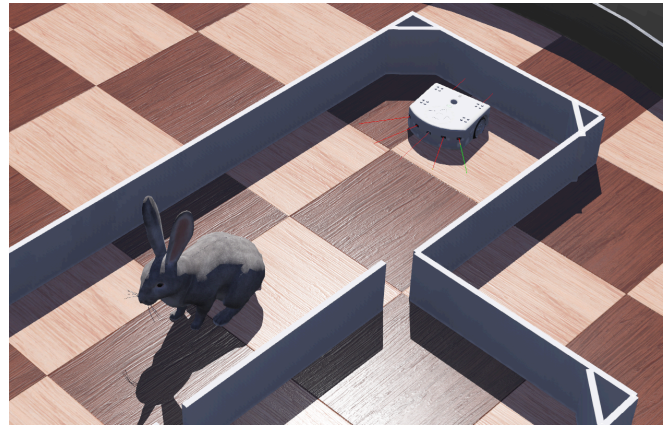


Figure 9: Parcours de simulation utilisé pour observer les effets de mémoire

## II.D RÉSULTATS OBTENUS

Les résultats suivants montrent la commande de la sortie  $y_2$  obtenue avec différentes valeurs de poids récurrents.

Sur la figure Figure 10, on observe que sans mémoire, la sortie  $y_2$  retourne rapidement à 1.0 dès que le capteur ne détecte plus l'obstacle, ce qui peut entraîner une perte de contrôle et une trajectoire erratique.

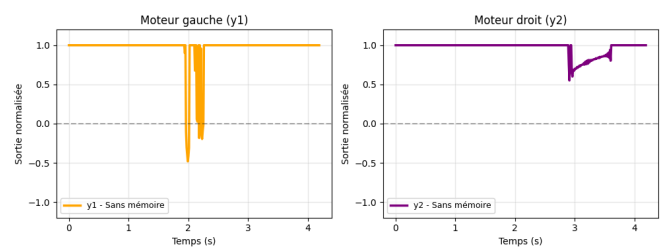


Figure 10: Sortie sans mémoire

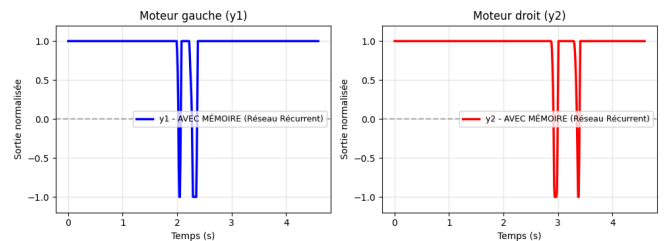


Figure 11: Effet d'une mémoire avec  $w_4 = w_5 = 1.5$



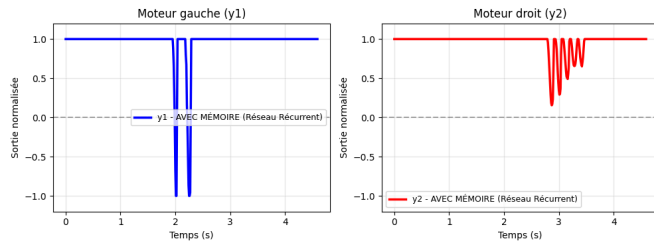


Figure 12: Effet d'une mémoire avec  $w_4 = w_5 = 1.0$

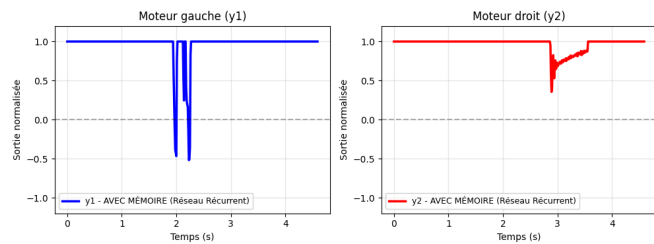


Figure 13: Effet d'une mémoire avec  $w_4 = w_5 = 0.5$

Sur les Figure 11, Figure 12 et Figure 13, on observe que l'introduction de la mémoire permet à la sortie  $y_2$  de rester activée plus longtemps même lorsque le capteur ne détecte plus l'obstacle. En revanche, l'introduction de la mémoire entraîne un effet oscillant dans la sortie  $y_2$ , apparenté au lag donné par la mémoire. Cette inertie donne un comportement moins vif et persistant.

Mettre un poids fort peut entraîner une persistance excessive, rendant thymio moins réactif à un nouvel obstacle. Le neurone reste activé trop longtemps.

Mettre un poids réduit sur la mémoire ( $w_4 = w_5 = 0.5$ ) permet de limiter l'effet d'inertie tout en réalisant un lissage sur la commande. D'après les observations que j'ai pu faire, ( $w_4 = w_5 = 0.2$ ) semble être un bon compromis pour garder une bonne réactivité et une mémoire.

### III FILTRE SPATIAL

#### III.A PRINCIPE ET OBJECTIF

Le filtre spatial permet de détecter des contrastes spatiaux dans les données des capteurs. L'objectif

est de distinguer un objet ponctuel d'un mur en analysant la distribution spatiale des activations. Le réseau utilise un mécanisme d'influence latérale : chaque neurone amplifie l'activation du capteur qui lui correspond tout en influençant les activations des capteurs voisins.

#### III.B ARCHITECTURE DU RÉSEAU TESTÉ

Le réseau implémenté suit la topologie de la Figure 6 du sujet. Il comporte :

- **5 neurones d'entrée** ( $x_1$  à  $x_5$ ) : capteurs de proximité avant du robot
- **5 neurones de sortie** ( $y_1$  à  $y_5$ ) : chacun détecte un pic spatial à une position donnée

Les équations d'activation pour chaque neurone de sortie sont :

$$y_1 = \text{sat}(4x_1 - 4x_2)$$

$$y_2 = \text{sat}(4x_2 - 2x_1 - 2x_3)$$

$$y_3 = \text{sat}(4x_3 - 2x_2 - 2x_4)$$

$$y_4 = \text{sat}(4x_4 - 2x_3 - 2x_5)$$

$$y_5 = \text{sat}(4x_5 - 4x_4)$$

où  $\text{sat}(x) = \text{clip}(x, 0, 1)$  est la fonction de saturation.

Les poids positifs (+4) amplifient le signal du capteur principal, tandis que les poids négatifs (-2 ou -4) inhibent les capteurs voisins. Les neurones de bord ( $y_1$  et  $y_5$ ) ont une influence plus forte (-4) car ils n'ont qu'un seul voisin.

#### III.C CAS D'USAGE TESTÉS

Plusieurs configurations d'entrée ont été testées pour valider le comportement du filtre :

1. **Objet ponctuel central** :  $x = [0, 0, 1, 0, 0]$ 
  - Seul le capteur central détecte l'objet
2. **Mur en biais** :  $x = [1.4, 1.2, 1.0, 0.8, 0.6]$

- Tous les capteurs détectent un obstacle à distances variables
3. **Obstacle étendu central** :  $x = [0, 1, 1, 1, 0]$
- L'objet est détecté sur trois capteurs centraux
4. **Obstacle latéral gauche** :  $x = [2.0, 1.5, 1.0, 0, 0]$
- L'objet est plus proche du côté gauche

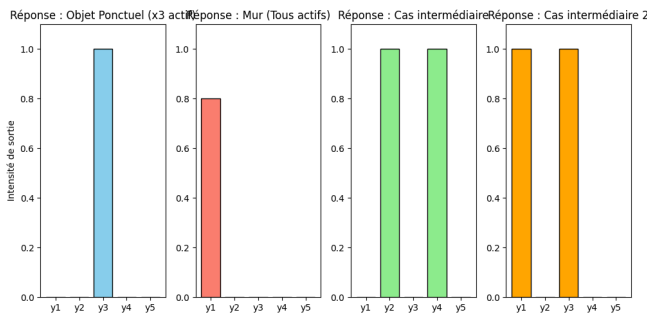


Figure 14: Résultats du filtre spatial sur différentes configurations d'obstacles

### III.D RÉSULTATS ET ANALYSE

Les résultats obtenus, illustrés par la Figure 14, montrent que le filtre spatial remplit correctement son rôle :

- **Objet ponctuel** : La sortie  $y_3$  (neurone central) est maximale (1.0), les autres sorties sont nulles. Le contraste spatial est parfaitement détecté.
- **Mur en biais** : Le neurone correspondant au point le plus proche ( $y_1$ ) présente une activation significative. Les gradients de distance créent un pic spatial localisé.
- **Obstacle étendu** : Plusieurs neurones centraux ( $y_2, y_3, y_4$ ) s'activent, indiquant une zone de détection large. L'absence d'activation sur les bords signale que ce n'est pas un mur complet.
- **Obstacle latéral** : Les neurones du côté correspondant ( $y_1, y_2$ ) s'activent fortement, permettant au robot d'identifier la direction de l'objet.

**Point important** : Dans le cas d'un mur parfaitement frontal où tous les capteurs détectent à la même distance ( $x = [1, 1, 1, 1, 1]$ ), les poids négatifs des voisins compensent exactement les poids

positifs centraux. Le résultat est une sortie nulle ou très faible sur tous les neurones, ce qui permet de distinguer un mur d'un objet.

Cette propriété d'influence latérale permet donc au réseau de détecter automatiquement les contrastes spatiaux sans utiliser de règles conditionnelles explicites.

Le code et les analyses de cette partie sont disponibles dans le notebook du TP6.

### III.E EXERCICE D'APPLICATION SUR WEBOTS

Le robot doit respecter deux contraintes comportementales contradictoires avec une architecture de Braitenberg :

1. **Attraction** vers un objet isolé (le robot doit se tourner vers lui).
2. **Arrêt** face à un mur (le robot ne doit pas avancer).

#### III.E.1 1. GESTION DE L'ATTRACTION

Pour gérer l'attraction vers un objet, j'ai utilisé le même filtre spatial que dans la partie précédente. Cependant, j'ai rencontré un problème : le robot était attiré vers l'objet mais aussi repoussé par les zones d'influence latérales générées par le filtre spatial.

En implémentant la topologie visible sur la Figure 15, j'ai constaté que les valeurs négatives générées par le filtre spatial pour les capteurs voisins étaient propagées aux moteurs, ce qui inversait le sens de rotation attendu et provoquait une répulsion.

J'ai ainsi ajouté ces poids suivants pour les connexions du filtre spatial vers les moteurs :

```
1 w_cote = 2.0
2 w_cote_ext = 2.0
3
4 w_front = 1.0
```

python

5 `w_cross = 1.0`

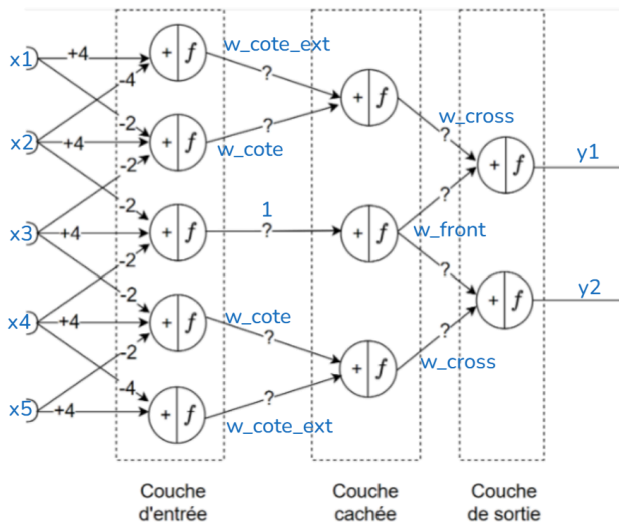


Figure 15: Topologie du réseau utilisé

Cette topologie permet de gérer l'attraction vers un objet, en prenant en compte les influences latérales.

L'attraction vers l'objet est visible sur la vidéo de la simulation disponible dans le dossier Partie 3 du TP6.

### III.E.2 2. GESTION DE L'ARRÊT FACE AU MUR

Une fois l'attraction réglée, le robot avançait vers le mur au lieu de s'arrêter. Le comportement est à réadapter pour que le robot puisse différencier un objet du mur. Les poids précédemment utilisés pour l'attraction devraient fonctionner mais j'ai remarqué des valeurs abérantes dans les entrées des capteurs latéraux. En effet, après la remarque d'un camarade, je n'avais pas pris en compte le fait que les capteurs latéraux étaient en arc de cercle. Il faut donc appliqué un poids plus fort pour ceux-ci ou de compenser les poids du filtre spatial.

Les nouvelles valeurs de ces points sont les suivantes :

1 `W_spatial = np.array([` python

```

2         [6.6, -4.0, 0.0, 0.0, 0.0],
3         # y1 : détecteur extrême gauche
4         [-3.2, 4.0, -2.0, 0.0, 0.0],
5         # y2 : détecteur gauche
6         [0.0, -2.0, 4.0, -2.0, 0.0],
7         # y3 : détecteur centre
8         [0.0, 0.0, -2.0, 4.0, -3.2],
9         # y4 : détecteur droite
10        [0.0, 0.0, 0.0, -4.0, 6.6]
11        # y5 : détecteur extrême droite
12    ])

```

Le rapport de valeur entre les capteurs latéraux et gauche/droite face au mur est d'environ 1.66.

Cette configuration devrait permettre au robot de s'arrêter face au mur mais un bug provoqué dans webot retourne une valeur nulle au capteur central lorsque le robot est face à un mur, ce qui empêche le comportement d'arrêt de se déclencher. J'ai essayé de compenser ce problème en augmentant les poids des capteurs latéraux mais cela n'a pas permis de régler le problème.

Ce comportement est visible sur la vidéo de la simulation disponible dans le dossier Partie 3 du TP6.

## IV CONCLUSION

*Pour conclure*, ce TP m'a permis de mieux comprendre les différentes topologies de réseaux de neurones artificiels et leur application à la robotique. J'ai pu expérimenter avec des architectures multicouches, récurrentes et des filtres spatiaux pour résoudre des problèmes d'évitement d'obstacles et de détection de contrastes spatiaux. J'ai également constaté l'importance de la configuration des poids et de la topologie du réseau pour obtenir le comportement souhaité.